# Test Plan — SCRUM-5: Implement Refund API with Full and Partial Refund Support

Status: DRAFT — pending human review. Not approved until a QA owner signs off.

## 1. Scope & Objectives

- **In scope:** refund eligibility checks, full and partial refund flows, refund amount validation, unique refund IDs, idempotency behavior, status lookup and lifecycle states.
- **Out of scope:** payment gateway settlement processing, finance reconciliation, UI flows, customer notifications, external accounting or ERP integration.
- **Objective:** confirm that only successful payments can be refunded, that refund amounts never exceed the remaining refundable balance, that duplicate requests are deduplicated, and that refund status can be queried reliably.

## 2. Gaps & Questions for the author

| # | Area | Finding (⚠️ / ❌) | Question to author |
|---|---|---|---|
| 1 | Refund eligibility | ⚠️ | Which non-successful payment states should be rejected (for example Pending, Failed, Cancelled)? |
| 2 | Refund amount rules | ⚠️ | Is the minimum allowed refund amount defined? Are there limits for the number of partial refunds per payment? |
| 3 | Idempotency contract | ⚠️ | What is the idempotency key TTL and what should happen when the same key is reused with a different payload? |
| 4 | Status lifecycle | ⚠️ | Which status transitions are valid, and are all statuses persisted for retrieval / audit? |
| 5 | Authorization & roles | ❌ | Who can create or query refund records: merchant, admin, support, or customer? |
| 6 | Environment & test data | ❌ | Which environments, feature flags, currencies, and seed data are required for test execution? |

## 3. Test Scenarios

| ID | Priority | Type | Scenario | Maps to |
|---|---|---|---|---|
| TS-01 | P0 | pos | Refund request succeeds for a previously successful payment with an amount within the remaining balance. | AC-1, AC-2, AC-4 |
| TS-02 | P0 | pos | Full refund request for the remaining balance is accepted and processed successfully. | AC-2, AC-4 |
| TS-03 | P0 | pos | Partial refund request is accepted and reduces the remaining refundable balance without exceeding it. | AC-2, AC-4 |
| TS-04 | P0 | neg | Refund request for a pending or failed payment is rejected. | AC-1, Gap-1 |
| TS-05 | P0 | neg | Refund request with an invalid or non-existent payment ID is rejected. | AC-8, Gap-6 |
| TS-06 | P0 | neg | Refund amount greater than the remaining refundable balance is rejected. | AC-3, AC-2 |
| TS-07 | P0 | neg | Refund request with amount 0 or negative value is rejected. | Gap-2 |
| TS-08 | P0 | pos | Multiple distinct refund requests receive unique refund IDs. | AC-4 |
| TS-09 | P0 | pos | Re-request with the same idempotency key returns the same outcome and does not create duplicate refunds. | AC-5 |
| TS-10 | P1 | pos | Refund status can be retrieved using the refund ID for Pending, Success, and Failed states. | AC-6, AC-7, Gap-4 |
| TS-11 | P1 | boundary | Refund process at the exact remaining balance boundary succeeds; any attempt above the balance fails. | AC-2 |
| TS-12 | P1 | neg | Same idempotency key with a different payload is rejected or handled according to policy. | AC-5, Gap-3 |

## 4. Test Data & Environment

- **Data:** successful payments, failed/pending/cancelled payments, existing refund history, payment records with partial refunds already applied, invalid payment IDs, duplicate idempotency keys.
- **Environment / flags:** QA or staging environment, refund API enabled, idempotency store active, payment records seeded, gateway mock or simulator enabled for status transitions.
- **Roles / permissions:** Merchant and admin roles should be explicitly confirmed; customer-facing or support actions are not defined in the ticket.

## 5. Risks & Assumptions

- **Assumptions made:** the refund endpoint is a backend API; the system keeps an idempotency store and refund records; status values are `Pending`, `Success`, and `Failed`.
- **Risks:** duplicate or race-condition requests, incorrect balance calculation after multiple partial refunds, delayed or inconsistent status transitions, lack of authorization guardrails.

## 6. Entry / Exit criteria

- **Entry:** refund API service is deployed to the target environment; test data is seeded; payment and refund stores are available; feature flags and dependency mocks are enabled.
- **Exit:** all P0 scenarios pass, no defect remains open for the primary refund path, all open questions are reviewed, and the plan is signed off by a QA owner.

## 7. Resource Allocation

- QA engineer: test plan review, scenario execution, defect triage
- Backend engineer: refund logic validation and defect analysis
- Product owner / business owner: requirement clarification on authorization and lifecycle rules

## 8. Timelines, Sprint Details

- **Sprint / target version:** Not specified in ticket; treated as TBD until the author confirms the planned release window.
- **Timeline:** No dates are specified in the ticket; schedule to be filled by the project team.

## Human Review Gate

- **I assumed:** the ticket is a backend API feature for refund creation and status lookup; the issue description is the source of truth; the product context is VWO / Bingify A/B testing platform.
- **I could not confirm:** authorization roles, exact non-successful payment states, idempotency TTL, status transition rules, and whether a minimum refund value or refund cap exists.
- **Open questions blocking sign-off:** Which non-successful payment states are rejected? What is the idempotency key policy? Which roles can perform refunds? Do we need a minimum refund or maximum number of partial refunds?
- **Approve, or edit, before I write test cases / automation.**